Report – IUM

2024/2025

Project by: Cutrò Riccardo, Precisvalle Alessandro



UNIVERSITÀ
DI TORINO

Introduction:

In this report we describe the work carried out for the IUM component of the IUM-TWEB project. The main task consisted of cleaning and analyzing data from a set of CSV files related to movies, actors, genres, languages, awards, and other attributes. The goal was to prepare the data for further use in a web platform and to extract relevant insights through data analysis.

We used Python and Jupyter Notebook as our main tools, leveraging libraries such as Pandas, NumPy, and Seaborn/Plotly for data manipulation and visualization. Each dataset was treated carefully to handle inconsistencies, missing values, and structural issues.

Technical Task 1: Data Cleaning and Preparation from CSV

a) Solution

The first step of the project involved cleaning and preparing multiple CSV datasets related to movies, actors, genres, languages, countries, awards, and other relevant information. The goal was to sanitize the input data, remove inconsistencies, and ensure referential integrity before populating the relational or NoSQL databases. The solution was implemented using Python and Pandas in Jupyter Notebooks. For each CSV file, we followed a similar pipeline:

1. **Loading the data**
   Each CSV was imported using pandas.read_csv(). We printed top records and dataset dimensions to get a first idea of the structure.
2. **Inspecting and correcting data types**
   Data types were adjusted using astype() to ensure consistency (e.g., converting IDs to Int64, ratings to float64, text fields to string type). We explicitly handled null values using fillna(pd.NA) or dropna() where necessary.
3. **Removing duplicates**
   Duplicates were checked using duplicated().sum() and removed with drop_duplicates() to avoid integrity issues when importing into the databases.
4. **Handling missing values**
   Some fields, like tagline or description, were allowed to remain empty. However, for critical fields (e.g., id, name), any row with missing data was removed using dropna(subset=[...]).
5. **Preparing table creation queries**
   A function execute_query() was used to run CREATE TABLE IF NOT EXISTS SQL scripts for each entity, defining schema constraints (e.g., CHECK, PRIMARY KEY).

This approach was modular and repeatable, and it ensured that all datasets could be imported without violating integrity constraints.

b) Issues

The main challenges encountered during the cleaning process included:

- Inconsistent or malformed data: Dates were sometimes stored as strings or contained missing values, and some numeric fields (like movie duration) were empty or non-numeric.
- Referential mismatches: Many files referenced id_movie, but some records referred to nonexistent movies. This was solved by filtering using a list of valid movie IDs.
- Duplicate records: Some files (especially actors, crews, posters) contained repeated entries that could cause duplication in queries or visualizations.
- Unreliable encoding: Certain CSVs had encoding issues due to special characters in names, which required specifying the correct encoding in read_csv().

c) Requirements

This solution fully complies with the original assignment objectives:

- Data was cleaned in a structured and reproducible way using Python and Pandas.
- Every CSV was converted into a valid table schema with proper constraints.
- Missing and duplicated data were handled carefully to ensure database integrity.
- The same method was applied across all datasets, making it easy to maintain and extend.

d) Limitations

Despite the robustness of the pipeline, some limitations remain:

- Hardcoded logic: The script expects column names to be consistent across all CSVs. If column names change, manual intervention is needed.
- Static filtering rules: Columns like rating, minute, or date are only checked for type and not semantic correctness (e.g., a rating of 15.0 may pass checks but be unrealistic).
- Scalability: The current method is suitable for small-to-medium datasets. For very large datasets, using chunked processing or SQL bulk loaders would be more efficient.

- Data lineage: No tracking is implemented for transformed records (e.g., logging what was dropped or changed), which limits auditability.

Technical Task 2: Database Connection Scripts

a) Solution

We developed two utility modules to manage database connections in Python:

- For PostgreSQL, we used SQLAlchemy and environment variables to create a reusable engine (get_db_engine()), execute queries (execute_query()), and fetch data into Pandas DataFrames (get_dataframe()).
- For MongoDB, we used PyMongo to define connection helpers (get_mongo_client() and get_mongo_db()), allowing centralized access to the reviews database.

Both scripts abstract connection logic for easier reuse across notebooks and ensure basic error handling.

b) Issues

- Required setting up environment variables correctly for PostgreSQL.
- Handled local MongoDB startup and occasional connection failures.

c) Requirements

The connection utilities ensure modular, reusable access to both databases, fulfilling the requirement of integrating analysis scripts with persistent storage.

d) Limitations

- Designed for local use only (no remote MongoDB/PostgreSQL configuration).
- Basic error reporting, no advanced logging or retries.

Technical Task 3: Data Analysis and Visualization Notebooks

a) Solution

We created three Jupyter notebooks to analyze and visualize the cleaned data stored in the PostgreSQL database. Each notebook focuses on a different aspect of the dataset:

- movies_analysis.ipynb explores general statistics on movies, including rating distributions and production years.
- genre_analysis.ipynb focuses on the frequency and characteristics of movie genres, identifying trends or dominant categories.
- quality_analysis.ipynb compares movie quantity and average rating across countries, with visualizations highlighting the relationship between production volume and quality.

All notebooks use the same modular setup: data is queried using the custom get_dataframe() function from the connection module, processed using Pandas, and plotted using Plotly for interactive visual output. Transformations include grouping, aggregation, joins, and normalization (e.g., log scaling) where necessary. Interactive plots include bar charts, scatter plots, and choropleth maps, designed to make the data easy to explore and interpret visually. Common techniques used include filtering null values, calculating averages and counts, and converting categorical data (e.g., country names to ISO codes) for mapping.

b) Issues

- Required careful joins and filtering to avoid mismatched or missing foreign keys.
- Some countries or genres were unrecognized in mapping libraries and had to be excluded.
- Choropleth maps required converting country names to ISO codes, which involved manual corrections or exclusions.
- Performance with large datasets could slow down interactivity in the notebook environment.

c) Requirements

These notebooks fully meet the assignment goal of analyzing the cleaned data and providing clear, interpretable visual insights. The code is modular, reusable, and readable, enabling further extensions or refinements with minimal effort.

d) Limitations

- Notebooks depend on local database access; they are not portable without configuration.
- Some filtering and transformation steps (e.g., log scaling) are tailored to this dataset and may need adjustment for future use.
- Analyses are exploratory, not predictive: no statistical testing or modeling was included.

CONCLUSIONS

This part of the project allowed us to apply data cleaning and analysis techniques on real-world data, reinforcing our skills in Python, Pandas, and SQL. We handled typical issues such as missing values and duplicates, producing a clean and consistent dataset.

The analysis notebooks offered useful visual insights and were structured for clarity and reusability. Overall, we are satisfied with the result, as the solution is modular, effective, and complements the backend work from the TWeb section.

DIVISION OF WORK

The work related to this IUM component was shared equally between both team members. Riccardo was primarily responsible for the data cleaning phase, including CSV inspection, type conversion, and the creation of database tables. Alessandro focused mainly on the data analysis part, which involved querying the database, processing data within Jupyter notebooks, and generating visualizations.

Both members collaborated on the overall design of the workflow and reviewed each other's contributions to ensure consistency, correctness, and alignment with the project objectives.

EXTRA INFORMATION

To run the notebooks:
PostgreSQL must be running locally and accessible via the credentials defined in the .env file. The tables will be created automatically by the cleaning scripts if they don't exist. The .env file must contain the following fields: DB_USERNAME, DB_PASSWORD, DB_HOST, DB_PORT, DB_NAME.

BIBLIOGRAPHY

- OpenAI. ChatGPT (Version 4.0). Language model used to assist with writing, code review, and content refinement. OpenAI, 2025. Available at: https://openai.com/chatgpt
- Pandas Documentation. Available at: https://pandas.pydata.org
- Plotly Express Documentation. Available at: https://plotly.com/python/plotly-express/
- W3Schools. W3Schools Online Web Tutorials. Retrieved June 25, 2025, from https://www.w3schools.com/